Polymorphism & Abstract Classes

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **trace** which version of a method runs through a superclass reference.
2. Be able to **use** `instanceof` to downcast safely.
3. Be able to **write** an abstract class with an abstract method.

Recap: overriding

```
public class Textbook extends Book {
  @Override
  public String toString() {
    return super.toString() + "\n - Course: " + this.courseCode;
  }
}
```

A `Textbook` replaces the `toString()` it inherited, and reuses `Book`'s inside it.

A `Book` variable, a `Textbook` object

```
Book b = new Textbook("Calculus, 9th Edition", "James Stewart", "MATH 221");
System.out.println(b);
```

Which `toString()` runs: `Book`'s, or `Textbook`'s?

A `Book` variable, a `Textbook` object

```
Book b = new Textbook("Calculus, 9th Edition", "James Stewart", "MATH 221");
System.out.println(b);   // Textbook's! The object decides, not the variable.
```

This is **polymorphism**: one call, and each object runs its own version.

One shelf, every kind of book

```
ArrayList<Book> shelf = new ArrayList<Book>();
shelf.add(calc);          // a Textbook
shelf.add(dune);          // a Novel
shelf.add(webster);       // a ReferenceBook

for (Book b : shelf) {
  System.out.println(b);   // each one runs its own toString
}
```

→ Step through a shelf in a visualization.

Eclipse Exercise

Get today's starter from Canvas.

TODO 1a. One `ArrayList<Book>` shelf, and one loop that prints every book.

TODO 1b. A second loop that renews every book.

Watch the dictionary!

Tophat Q1

A check on what a `Book` variable can call.

→ Open Tophat to vote.

The variable decides what you can call

```
Book b = new Textbook("Calculus, 9th Edition", "James Stewart", "MATH 221");
b.getCourseCode();   // The method getCourseCode() is undefined for the type Book
```

The **variable's type** decides which methods you can *call*. The **object** decides which version *runs*.

Downcasting

```
Textbook t = (Textbook) b;    // "trust me, it's a Textbook"
t.getCourseCode();            // compiles now
```

Going up is automatic: `Book b = calc;` is an **upcast**.

Going down needs a cast: `(Textbook) b` is a **downcast**.

...as long as it really is a `Textbook`

```
Book b = new Novel("Dune", "Frank Herbert", "Science Fiction");
Textbook t = (Textbook) b;   // compiles... then crashes
```

```
ClassCastException: class Novel cannot be cast to class Textbook
```

→ Watch it crash in a visualization.

Check first with `instanceof`

```
if (b instanceof Textbook) {
  Textbook t = (Textbook) b;   // safe: we just checked
  System.out.println(t.getCourseCode());
}
```

The same move as Wednesday's `equals`.

Eclipse Exercise

TODO 2. How many MATH 221 textbooks are on the shelf? Count them in `main`.

How long does a book lend for?

A `Textbook` : 7 days (it's on reserve). A `Novel` : 30. A `ReferenceBook` : 0.

A plain `Book` ? **There's no right answer.** Let's make every book in our library *some kind* of book, not just a `Book` .

Abstract classes

```
public abstract class Book {
  public abstract int getLoanLength();   // no body: each kind decides
}
```

An **abstract** method has no body.

A class with one must be **abstract** -- and `new` can't make one.

Every kind fills it in

```java
public class Textbook extends Book {
  @Override
  public int getLoanLength() {
    return 7;   // on reserve
  }
}
```

A subclass that skips it gets: *The type Textbook must implement the inherited abstract method Book.getLoanLength()*

Eclipse Exercise

TODO 3a. Make `Book` abstract, with an abstract `getLoanLength()`. `renew()` should use it.

TODO 3b. Fill in `getLoanLength()` for each kind of book.

TODO 3c. Fix `Main` -- what kind of book are the lecture notes?

Tophat Q2

A check on abstract classes.

→ Open Tophat to vote.

Questions?